Spotnana IITK

Online assessment questions, written from memory. Everyone got the same set of three. Each question has at most 3 practice equivalents and the key idea.

Q1. Shortest walk covering all required nodes in a tree

QUESTION (FROM MEMORY)

An unweighted tree and a set of required vertices are given. Find the length of the shortest path that visits all the required vertices.

It is not a direct BFS/DFS: when the path backtracks, the backtracked steps also count toward the path.

Example: the path 1-2-3-2-4 covers the required vertices. Answer: 5.

Equivalents

Problem	Match	Why it maps
Codeforces 592D: Super M	**Exact**	Unweighted tree, set of required cities, start anywhere and end anywhere, backtracking counts.
LeetCode 1443: Minimum Time to Collect All Apples in a Tree	**Part 1**	Prune subtrees with no required node; a full round trip over the remaining tree costs 2 × edges.
LeetCode 1245: Tree Diameter (premium)	**Part 2**	Longest path in the pruned tree, which is the part you walk only once.

Key idea

Repeatedly delete leaves that are not required. The remaining minimal subtree has **E** edges. Because backtracking counts, a walk that returns to its start covers every edge twice: 2E. The walk may end anywhere, so the longest path in that subtree is walked only once. **Answer = 2E − D**, where D is the diameter of the pruned tree.

Check with the example 1-2-3-2-4: E = 3 and D = 2, so the walk is 4 edges, which is 5 nodes. The answer of 5 suggests the problem counts nodes (edges + 1). Confirm against the sample.

Q2. Train then execute: minimum days (or −1)

QUESTION (FROM MEMORY)

There are N ML algorithms. Each algorithm has a training time and must be fully trained before it can be executed.

A list of M days is given, saying which algorithm can be executed on which day (the time slots in which each task must be completed).

Find the minimum number of days in which all algorithms can be trained and executed, or return −1 if it is not possible.

Equivalents

Problem	Match	Why it maps
LeetCode 3048: Earliest Second to Mark Indices I	**Exact**	nums[i] is the training time; changeIndices says which index can be marked (executed) on each second.
Codeforces 732D: Exams	**Exact**	Each day allows passing one specific exam; each exam needs a_i days of preparation first.
LeetCode 1482: Minimum Number of Days to Make m Bouquets	**Pattern**	Binary search on the answer with a greedy feasibility check.

Key idea

The answer is monotonic: if everything fits in d days, it also fits in d + 1. Binary search on d.

check(d):

- For every algorithm, take its **last** allowed execution day within [1, d]. Executing as late as possible leaves the most free days for training. If some algorithm has no such day, return false.
- Scan days 1 to d. On an ordinary day, free += 1.
- On an algorithm's chosen day, require free ≥ train[i], then free −= train[i].

Complexity: O((N + M) log M).

Q3. k-th largest OR over all subarrays

> **QUESTION (FROM MEMORY)**
>
> An array is given. Take the bitwise OR of every possible contiguous subarray. Return the k-th largest of these OR values.
>
> The brute force (generating every subarray's OR) exceeded time or memory on 5 hidden test cases.

Equivalents

Problem	Match	Why it maps
LeetCode 898: Bitwise ORs of Subarrays	**Core trick**	At most ~31 distinct OR values end at each index, so all subarray ORs fit in O(31·n).
LeetCode 3171: Find Subarray With Bitwise OR Closest to K	**Same trick**	Same compressed per-index OR set, used to answer a query over all subarrays.
LeetCode 1918: Kth Smallest Subarray Sum (premium)	**k-th pattern**	Finding the k-th value over all O(n²) subarrays without listing them.

Why the brute force fails

There are n(n + 1)/2 subarrays, about 5×10^9 for $n = 10^5$. Storing them in a vector or heap exceeds memory, and even generating them exceeds time.

Key idea

For a fixed right end r, extending the subarray to the left only adds bits, so the OR values form a chain with at most ~31 distinct values (for 30-bit inputs). Keep (value, count) pairs per index:

`cur = {(a[r], 1)} ∪ {(v | a[r], c) for (v, c) in prev}, merging equal values`

- **k-th largest with duplicates:** collect all pairs (at most ~31n), sort by value descending, and add counts until the running total reaches k.
- **k-th largest distinct:** build the LeetCode 898 set, sort it, and take the k-th element.
- k can be as large as n², so use 64-bit integers.